VPS-DEPLOYMENT

Fresh VPS to production: firewall, Caddy + HTTPS, WireGuard tunnel to JeMPI, and JeMPI deployment

2026-08-28

- OpenHIE Stack — VPS Deployment
  - 0. Topology
  - 1. Provision & first access
  - 2. Host preparation
  - 3. Firewall & required ports
  - 4. DNS (domain mode only)
  - 5. WireGuard tunnel to JeMPI
  - 6. Deploy JeMPI (on its own VPS)
  - 7. Start the OpenHIE stack
  - 8. OpenHIM root password
  - 9. Realm + realm public key
  - 10. Point the Console at the public API (domain mode)
  - 11. Onboard hospitals
  - 12. Service management & boot persistence
  - 13. Deployment verification
  - 14. Raw-IP test profile notes (`vps-ip`)
  - 15. Full reset — destructive

OpenHIE Stack — VPS Deployment

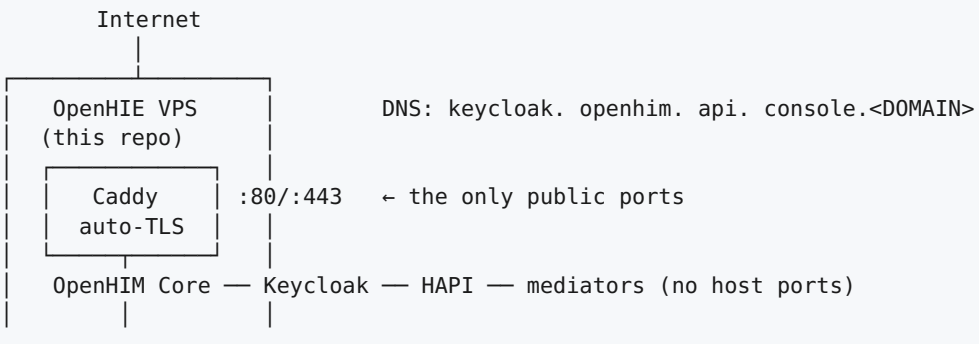
End to end: a fresh VPS with only an OS → an HTTPS-fronted OpenHIE stack, a WireGuard tunnel to JeMPI (or a public-port fallback), and — if you are also standing up the MPI — a JeMPI deployment on its own host.

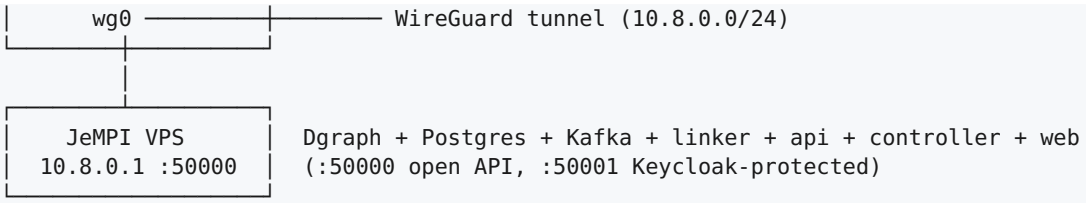
This assumes you have already read `SETUP-LINUX.md` (the per-step script / manual / GUI detail is there; this document is the VPS wrapper around it). Every step that has a script also states the manual + GUI path.

`<like-this>` = placeholder.

---

0. Topology





Two deployment modes for the OpenHIE VPS:

Mode	Compose profile	Front door	Use
Domain + HTTPS	`vps` (`caddy`)	`:443` on 4 subdomains of `$DOMAIN`	the real deployment
Raw-IP test	`vps-ip` (`caddy-ip`)	`:80` HTTP, Keycloak only	short test on a trusted network / VPN
local dev	none	direct host ports	see `SETUP-LINUX.md`

Run **one** profile at a time — both Caddy services bind port 80.

---

1. Provision & first access

- Ubuntu 22.04 LTS or Debian 12, `amd64`, ≥ 4 vCPU / 8 GB RAM / 40 GB SSD.
- A public IPv4 address. For domain mode, DNS control for a zone you own.

```
ssh root@<vps-ip>
# create an admin user (skip if the provider already made one)
adduser deploy
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy
# reconnect as deploy; everything below is as deploy
```

SSH hardening

`/etc/ssh/sshd_config.d/hardening.conf`:

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
```

```
sudo systemctl reload ssh
```

Confirm key login still works from a second terminal before closing the first.

---

2. Host preparation

Option A — bootstrap script

Copy just `keycloak/deploy-vps.sh` to the VPS (or `curl` it) and run:

```
DOMAIN=hie.example.org ./deploy-vps.sh        # domain + HTTPS mode
#   or
VPS_IP=203.0.113.10 ./deploy-vps.sh           # raw-IP test mode
```

It installs any missing prerequisites (`docker.io`, `docker-compose-v2`, `git`, `jq`, `curl`, `openssl`, `wireguard`), `systemctl enable --now docker`, adds you to the `docker` group, clones the repo to `/opt/openhie`, seeds the four mediator `.env` files from their examples, creates the placeholder `openhim-config/realm-public.pem`, and writes the **root `.env`** (`DOMAIN=` or `VPS_IP=`/`KEYCLOAK_PUBLIC_URL=`/`KEYCLOAK_ISSUER=`). It stops **before** `docker compose up` and prints the remaining manual steps. Re-runnable; it never overwrites an existing `.env`.

Option B — manual

```
sudo apt-get update && sudo apt-get -y upgrade
sudo apt-get install -y ca-certificates curl git jq openssl python3 wireguard ufw
curl -fsSL https://get.docker.com | sh
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER" && newgrp docker

sudo mkdir -p /opt/openhie && sudo chown "$USER:$(id -gn)" /opt/openhie
cd /opt && git clone https://github.com/ETech-Nepal/openhim.git openhie && cd openhie

for d in mediator admin-mediator patient-portal-mediator fhir-intake-mediator; do
  cp "$d/.env.example" "$d/.env"
done
mkdir -p openhim-config
[ -f openhim-config/realm-public.pem ] || touch openhim-config/realm-public.pem
chmod 600 */.env openhim-config/realm-public.pem
```

Root `.env` (both options must end up with this)

Domain mode — `/opt/openhie/.env`:

```
DOMAIN=hie.example.org
KEYCLOAK_PUBLIC_URL=https://keycloak.hie.example.org
KEYCLOAK_ISSUER=https://keycloak.hie.example.org/realms/openhie
```

Raw-IP mode — `/opt/openhie/.env`:

```
VPS_IP=203.0.113.10
KEYCLOAK_PUBLIC_URL=http://203.0.113.10
KEYCLOAK_ISSUER=http://203.0.113.10/realms/openhie
```

`DOMAIN` must be the bare zone (no scheme, no subdomain, no trailing dot); `Caddyfile` builds `keycloak.{$DOMAIN}` etc. from it. `KEYCLOAK_PUBLIC_URL` becomes Keycloak's `KC_HOSTNAME` (the `iss` of every token minted); `KEYCLOAK_ISSUER` becomes OpenHIM's `authentication_jwt_issuer`. They must be identical apart from the `/realms/openhie` suffix — any mismatch and every transaction request `401`s.

---

3. Firewall & required ports

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
```

Domain mode (`vps`)

```
sudo ufw allow 80/tcp     # ACME HTTP-01 + HTTPS redirect
sudo ufw allow 443/tcp    # Caddy
sudo ufw enable
```

Do not open 5000 / 8080 / 9000. Compose still publishes them on all interfaces and Caddy does **not** remove them — `ufw` is the only thing keeping them off the internet. Their TLS-terminated equivalents are `openhim.`, `api.`, `console.<DOMAIN>` on 443. `5001`, `8081`, `8083` are already loopback-only in Compose. Never publish `5432`, `27017`, or any mediator port.

Raw-IP test mode (`vps-ip`)

```
sudo ufw allow 80/tcp                                          # Keycloak via caddy-ip
sudo ufw allow from <admin-ip> to any port 5000 proto tcp      # OpenHIM tx (self-signed)
sudo ufw allow from <admin-ip> to any port 8080 proto tcp      # OpenHIM API
sudo ufw allow from <admin-ip> to any port 9000 proto tcp      # Console
sudo ufw allow <wg-port>/udp                                   # WireGuard, if this VPS is a
    peer
sudo ufw enable
```

---

4. DNS (domain mode only)

Create **four A records**, all pointing at the VPS public IP, and let them resolve **before** the first `--profile vps up` — Caddy requests certificates on boot over HTTP-01, and an unresolved name fails ACME (and repeated failures hit Let's Encrypt rate limits).

Record	Proxies to (internal)	For
`keycloak.<DOMAIN>`	`keycloak:8080`	token endpoint + admin console
`openhim.<DOMAIN>`	`openhim-core:5001`	client traffic: `/fhir`, `/ingest`
`api.<DOMAIN>`	`openhim-core:8080` (self-signed, verify skipped)	Core API for the Console
`console.<DOMAIN>`	`openhim-console:80`	Console UI

- **GUI:** in your DNS provider's console add four **A** records (`keycloak`, `openhim`, `api`, `console`) → VPS IP.
- **CLI verify:** `dig +short keycloak.<DOMAIN> openhim.<DOMAIN> api.<DOMAIN> console.<DOMAIN>` — all return the VPS IP.

Source: `Caddyfile`.

---

5. WireGuard tunnel to JeMPI

The mediators reach JeMPI at `JEMPI_BASE_URL` in `mediator/.env` **and** `admin-mediator/.env`. Two ways:

Option A — public port (simplest; test data only)

```
sed -i 's#^JEMPI_BASE_URL=.*#JEMPI_BASE_URL=http://<jempi-vps-ip>:50000/JeMPI#' \
  mediator/.env admin-mediator/.env
curl http://<jempi-vps-ip>:50000/JeMPI/countRecords     # must return JSON
```

Unencrypted, no auth on `:50000`. Restrict `:50000` on the **JeMPI** VPS firewall to this OpenHIE VPS's IP:

```
# on the JeMPI VPS
sudo ufw allow from <openhie-vps-ip> to any port 50000 proto tcp
```

Option B — WireGuard tunnel (recommended; keeps the default `10.8.0.1`)

On the JeMPI VPS (the tunnel server), if it is not already a WireGuard peer:

```
sudo apt-get install -y wireguard
wg genkey | sudo tee /etc/wireguard/server.key | wg pubkey | sudo tee /etc/wireguard/server.pub
```

`/etc/wireguard/wg0.conf` on the JeMPI VPS:

```
[Interface]
Address = 10.8.0.1/24
ListenPort = <wg-port>
PrivateKey = <contents of /etc/wireguard/server.key>

[Peer]                              # the OpenHIE VPS
PublicKey = <openhie-vps public key>
AllowedIPs = 10.8.0.2/32
```

```
sudo systemctl enable --now wg-quick@wg0
sudo ufw allow <wg-port>/udp
# allow the tunnel to reach the JeMPI containers on the docker network
sudo sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-jempi.conf
```

On the OpenHIE VPS (the tunnel client):

```
wg genkey | tee ~/wg.key | wg pubkey | tee ~/wg.pub    # give the .pub to the JeMPI VPS peer
        block
sudo tee /etc/wireguard/wg0.conf >/dev/null <<EOF
[Interface]
Address = 10.8.0.2/24
PrivateKey = $(cat ~/wg.key)

[Peer]
PublicKey = <jempi-vps server.pub>
Endpoint = <jempi-vps-ip>:<wg-port>
AllowedIPs = 10.8.0.0/24
PersistentKeepalive = 25
```

```
EOF
rm ~/wg.key

sudo systemctl enable --now wg-quick@wg0
curl http://10.8.0.1:50000/JeMPI/countRecords          # from the host — must return JSON
```

Keep `JEMPI_BASE_URL=http://10.8.0.1:50000/JeMPI` (the default) in both `mediator/.env` and `admin-mediator/.env`.

Container can't reach `10.8.0.1` even though the host can — masquerade the Docker subnet out the tunnel (the containers reach `10.8.0.1` through Docker NAT, which is not routed into `wg0` by default):

```
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/12 -o wg0 -j MASQUERADE
sudo sysctl -w net.ipv4.ip_forward=1
# persist the iptables rule
sudo apt-get install -y iptables-persistent && sudo netfilter-persistent save
```

Verify from inside a container **after step 7**:

```
docker exec mediator wget -qO- http://10.8.0.1:50000/JeMPI/countRecords
```

> You may run the tunnel **and** point `JEMPI_BASE_URL` at the public IP — both reach the same JeMPI. Use whichever your firewall policy allows.

---

6. Deploy JeMPI (on its own VPS)

JeMPI is a **separate open-source project** (`jembi/JeMPI`); it is **not** in this repo's `docker-compose.yml`. This repo only carries the **linker configuration** to apply to a running JeMPI (`jempi-config/`). Deploy JeMPI on a dedicated host and then apply that config.

6a. JeMPI host requirements

JeMPI's stack is heavier than the OpenHIE stack: Dgraph (graph DB, alpha + zero), PostgreSQL, Kafka + Zookeeper, and the JeMPI services (`api`, `controller`, `linker`, `bootstrapper`, `web`). Plan for **≥ 4 vCPU / 16 GB RAM / 100 GB SSD**; confirm against the JeMPI release you deploy. **Requires confirmation** — sizing is not defined in this repo.

6b. Get and deploy JeMPI

Follow the JeMPI project's own deployment docs. For the Docker-Swarm ("docker stack") deployment referenced in `DEMO.md`:

```
# on the JeMPI VPS
git clone https://github.com/jembi/JeMPI.git ~/JeMPI
cd ~/JeMPI/devops/linux/docker
# build images and bring the stack up per JeMPI's scripts, e.g.
#   ./d-stack-1-build.sh        # build images
#   ./d-stack-2-deploy.sh       # docker stack deploy
# exact script names vary by JeMPI version — use the ones in that directory.
```

`DEMO.md` also documents a recovery script on that host: `~/JeMPI/devops/linux/docker/d-stack-3-reboot.sh` — restarts JeMPI services without a full image rebuild, **and resets JeMPI's data store**. Use it only when the linker is stuck (reads work, writes hang) and you can afford to lose the data.

Expose:

- `:50000` — the open JeMPI API (no Keycloak). This deployment uses `:50000`.
- `:50001` — Keycloak-protected API. Production target (`FHIR_MEDIATOR_IMPLEMENTATION.md` §9); **not yet wired** into the mediators.

Reach it from the OpenHIE VPS either over the WireGuard tunnel (`http://10.8.0.1:50000/JeMPI`) or the public IP (`http://<jempi-vps-ip>:50000/JeMPI`).

6c. Apply this repo's linker configuration

From `jempi-config/README.md`. In JeMPI's `data-config` directory:

```
# 1. back up the stock config, then copy this repo's version over it
cp config-reference-link-dp.json config-reference-link-dp.json.bak
#    ...copy openhie/jempi-config/config-reference-link-dp.json here...

# 2. point config.json at it (default already does)
./use.sh config-reference-link-dp.json        # recreates config.json + config-api.json symlinks

# 3. set the linker decision threshold (env vars on the LINKER service, NOT in the json)
#    stock is 0.65 / 0.1 ; agreed policy: auto-link >= 0.75, hold 0.65-0.75 for review
LINKER_MATCH_THRESHOLD=0.75
LINKER_MATCH_THRESHOLD_MARGIN=0.10
#    edit these wherever your stack injects linker env
#    (e.g. devops/linux/docker/conf/stack/docker-stack-low-1.yml), then redeploy the linker.

# 4. REGENERATE THE DGRAPH SCHEMA — this config adds a dob index (@index(exact,trigram)).
#    A plain linker restart does NOT apply it, and eq(dob) / match(dob,3) will fail.
#    With an empty DB the clean path is to tear the JeMPI stack down and bring it back up
#    so the bootstrapper re-applies the schema from config-api.json:
#      docker stack rm <stack> ; (wait for full teardown) ; ./d-stack-2-deploy.sh
#    If your stack has a dedicated schema/bootstrapper step, re-run that instead.
```

6d. What the config changes (vs stock JeMPI)

	Stock	This repo
Deterministic rules	`eq(national_id)`, `eq(name)+eq(phone)`	`eq(national_id)`, **or** `eq(given_name)+eq(family_name)+eq(dob)+eq(phone_number)+eq(gender)`
Probabilistic blocking	name+name, name+city, family+city, phone, nid	name+name, name+**dob**, family+**dob**, phone, nid
`dob`	scoring-only (no index)	indexed on golden + interaction (schema change)

`eq(name)+eq(phone)` is unsafe here because the phone is very often the placeholder `9800000000`; requiring dob+gender makes the exact-match key safe. National ID stays optional — JeMPI skips a rule clause whose field is blank, so a patient with no NID still links via name/dob/phone.

6e. Verify JeMPI

```
BASE=http://10.8.0.1:50000/JeMPI          # or http://<jempi-vps-ip>:50000/JeMPI
curl -s $BASE/countRecords | jq
curl -s $BASE/configuration | jq '.rules.link'                                    # 2
        deterministic clauses + dob (not city) in probabilistic
curl -s $BASE/configuration | jq '.demographicFields[] | select(.fieldName=="dob")'  # now
        indexed
```

---

7. Start the OpenHIE stack

Change default secrets first (domain mode — public admin surfaces)

- `KC_BOOTSTRAP_ADMIN_PASSWORD` in `docker-compose.yml` (ships `admin`).
- HAPI DB / Mongo credentials in `docker-compose.yml` (ship `admin`/`admin`) — recommended for production.

Bring it up

```
cd /opt/openhie
docker compose --profile vps up -d --build           # domain mode
#   or
docker compose --profile vps-ip up -d --build        # raw-IP test mode
docker compose ps
docker compose logs -f caddy                          # domain mode: "certificate obtained
        successfully" per name
```

`caddy` (or `caddy-ip`) must appear in `docker compose ps`; if it is missing you omitted the `--profile` flag. Every later Compose command that should keep Caddy running needs the flag too (`docker compose --profile vps stop|start|up`). Certificates live in the `caddy-data` volume and survive `stop`/`start` and recreation.

Mediators crash-loop until steps 8 and 9 are done — expected.

---

8. OpenHIM root password

Follow `SETUP-LINUX.md` step 3 (GUI to set it, script or manual to propagate), substituting the URL:

- Console: `https://console.<DOMAIN>` (domain) or `http://<vps-ip>:9000` (raw-IP; open the port to your admin IP only).
- Trust the cert first at `https://api.<DOMAIN>/heartbeat` or `https://<vps-ip>:8080/heartbeat`.

```
NEWPW='<the-password-you-set>'
for d in mediator admin-mediator patient-portal-mediator fhir-intake-mediator; do
  sed -i "s/^OPENHIM_PASSWORD=.*/OPENHIM_PASSWORD=${NEWPW}/" "$d/.env"
done
docker compose up -d --force-recreate mediator admin-mediator patient-portal-mediator fhir-
        intake-mediator
```

---

9. Realm + realm public key

`SETUP-LINUX.md` steps 4 and 5, with one VPS difference: Keycloak's `8083` stays bound to `127.0.0.1`, so the `keycloak/*.sh` scripts keep working **unchanged on the VPS itself** against `http://localhost:8083`. Only the advertised issuer changes — with `KC_HOSTNAME` set, `.well-known/openid-configuration` reports the public issuer even over loopback, and it must match `KEYCLOAK_ISSUER`.

```
# realm (auto-imported, or ./keycloak/setup.sh)
curl -fsS http://localhost:8083/realms/openhie/.well-known/openid-configuration | jq .issuer
# → https://keycloak.<DOMAIN>/realms/openhie   (domain mode)

# realm public key into OpenHIM
./keycloak/extract-realm-pubkey.sh
docker compose --profile vps up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

After any change to the root `.env` (`DOMAIN` / `KEYCLOAK_*`):

```
docker compose --profile vps up -d --force-recreate keycloak openhim-core
./keycloak/extract-realm-pubkey.sh
docker compose --profile vps up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

---

10. Point the Console at the public API (domain mode)

The Console is a browser app — it calls the Core API from the user's browser, so it must be told the public API host. Set on the live `openhim-console` service (the commented block in `docker-compose.yml` shows the shape):

```
    environment:
      OPENHIM_CONSOLE_PROTOCOL: "https"
      OPENHIM_CORE_MEDIATOR_HOSTNAME: "api.<DOMAIN>"
      OPENHIM_MEDIATOR_API_PORT: "443"
```

```
docker compose --profile vps up -d --force-recreate openhim-console
```

Left at defaults, the Console loads at `https://console.<DOMAIN>` but every API call fails (looks like a login problem).

---

11. Onboard hospitals

`SETUP-LINUX.md` step 7 (script / manual / GUI). On the VPS itself the scripts run unchanged. From an admin machine **outside** the VPS, point them at the public Keycloak host:

```
KEYCLOAK_URL=https://keycloak.<DOMAIN> ./keycloak/onboard-hospital.sh 3130400221 "Bharatpur
        Hospital"
```

Client traffic (tokens + transactions), domain mode, no `-k` needed:

```
KEYCLOAK_URL=https://keycloak.<DOMAIN> TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)
curl -sS -X POST https://openhim.<DOMAIN>/fhir/Patient \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/fhir+json' \
  -d '{"resourceType":"Patient","name":[{"family":"SHRESTHA","given":["KAMANA"]}],
       "gender":"female","birthDate":"2001-06-25"}' | jq '{id,identifier,meta}'
```

`/ingest` is the same host: `https://openhim.<DOMAIN>/ingest`.

---

12. Service management & boot persistence

Every service has `restart: unless-stopped` and Docker is `systemctl enable`d, so the stack comes back after a reboot. Because the profile flag is not remembered, either always run `docker compose --profile vps up -d` after a reboot, or add a tiny systemd unit:

`/etc/systemd/system/openhie.service`:

```
[Unit]
Description=OpenHIE stack
Requires=docker.service
After=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/openhie
ExecStart=/usr/bin/docker compose --profile vps up -d
ExecStop=/usr/bin/docker compose --profile vps stop
User=deploy

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable openhie.service
```

Verify: `sudo reboot`; after reconnect `docker compose --profile vps ps` → all `Up`; WireGuard: `sudo wg show` shows a live handshake.

---

13. Deployment verification

```
docker compose --profile vps ps            # all Up; fhir healthy after ~1 min
sudo wg show                                # (Option B) recent handshake with the JeMPI peer
docker exec mediator wget -qO- "$(grep -h '^JEMPI_BASE_URL=' mediator/.env | cut -d= -f2-)/
        countRecords"

KEYCLOAK_URL=https://keycloak.<DOMAIN> TOKEN=$(./keycloak/get-jwt-client-token.sh <nhfr>)
curl -sS -X POST https://openhim.<DOMAIN>/fhir/Patient \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/fhir+json' \
  -d '{"resourceType":"Patient","name":[{"family":"DEMO","given":["VPS"]}],
       "gender":"unknown","birthDate":"1990-01-01"}' | jq '{id,identifier}'
```

Expected: `201/200` with an opaque `id` and both `nepal-health-id` and `jempi-golden-uid`. Console → **Transactions** → the request → **Orchestrations** shows `JeMPI register` → `HAPI read Patient by Health ID` → `PUT HAPI Patient`.

From **off-box**, confirm the loopback ports are closed: `nc -vz <vps-ip> 5000` / `8080` / `9000` must all **fail** (only 80/443 open).

---

14. Raw-IP test profile notes (`vps-ip`)

- Plain **HTTP** for Keycloak via `caddy-ip` — credentials and tokens are not protected. Trusted network / WireGuard clients only. Move to the `vps` profile before anything real.
- OpenHIM keeps its direct `:5000` (self-signed) — transaction tests go to `https://<vps-ip>:5000` with `-k`.
- `KEYCLOAK_URL=http://<vps-ip>` for every `keycloak/*.sh` call.
- Continue with WireGuard/JeMPI (§5), OpenHIM password (§8), realm key (§9), hospital onboarding (§11).

---

15. Full reset — destructive

```
docker compose --profile vps down -v --rmi all
```

Drops Keycloak H2, OpenHIM Mongo, HAPI + `admin_review` Postgres, **and Caddy's certificates**. Survives: `keycloak/keys/*.pem`, `*/.env` (root pw now wrong), stale `realm-public.pem`, `hapi-config/`, `jempi-config/`. JeMPI (separate host) is untouched. Before repeating a wipe, make sure DNS still resolves (§4) — Let's Encrypt rate-limits repeated issuance for the same names. Rebuild: §7 → §8 → §9 → §11.